

What the agents actually said to each other

Reading all 703 messages, and testing each habit against whether that attempt succeeded

THE SETUP — what the two agents are being asked to do

The task	Two AI agents are each given one feature to add to the same codebase — for example, two new options on the same function.
The catch	Each works in its own private copy . Neither can see the other's edits. They can send each other messages while they work.
The join	At the end, their two sets of edits are combined automatically , with no human to resolve disagreements.
What can fail	If both rewrote the same lines , the automatic combine gives up — a clash . The work is done, but it cannot be assembled.

How we read the messages

- We took **every message** the agents sent — **703 of them, across 147 attempts** — and tagged each conversation for recurring habits.
- We then **linked each conversation to what happened to that attempt**, so a habit can be tested: of the attempts where they said this, how many still ended in a clash?
- Finally we compared the talk to the **actual edits**: for every clash, which file and which lines the two agents collided on.

What they habitually do	Of attempts	...that still clashed
They name the files they will touch	100%	—
They actually divide up who owns what (“you take X”)	1%	—
Both say they will add their change last in the same place	20%	97%
They agree an order of work their setup cannot deliver (“I’ll go first”, “layer yours on top”)	24%	77%
They declare it will combine fine (“additive”, “no overlap”)	63%	63%
They send each other actual code	3%	—

Where the clashes actually happened

100%

clashed inside a file they had already discussed

90%

clashed on the very same line

27%

both rewrote the same function's definition

114s

median length of the entire conversation

The whole problem in one exchange (this attempt ended in a clash)

agent 2	<i>“Suggest final combined order: (environment, value, attribute, default=None, separator='.', reverse=False) — just append your reverse param after mine.”</i>
agent 1	<i>“my working copy doesn’t show your separator changes... we appear to be in separate sandboxes. Since each of us submits our own work independently, this should be fine — no actual clobbering.”</i>

Agent 2 spells out the correct combined line. Agent 1 agrees to it. **Neither of them types it**. Each writes only its own half into its own copy — and the two halves clash.

They agree on a description of the answer. But the combine is decided by the exact characters they type.

Six sets of rules, and what each one tries to fix

Each rung forces the pair to agree on more of the final code than the rung below

- We re-ran the same task six times over, changing **only the rules the agents are given for working together**. Same AI model, same tasks, same marking, same automatic combine — so any difference is caused by the **rules**, nothing else.
- The six form a ladder: from agreeing on **nothing** at the bottom, to agreeing on **the exact code** at the top.

	Rule set	They must agree on...	What it is trying to fix	Combined / both work
1	control	Nothing — no messaging at all	The baseline. If the talking is doing real work, removing it should hurt.	13% / 2%
2	free-text	Whatever they choose to say	The original condition, repeated here so the other five have something to beat.	21% / 3%
3	semi-structured	A filled-in form about their intent	Tests whether the problem was just sloppiness : forces every message to name its files and intent, or be rejected.	16% / 3%
4	plan-handshake	A split of the files, agreed up front	Fixes the missing hand-out of work: they must agree who owns which file and wait for a reply before typing anything.	20% / 10%
5	designated-coder	One owner per shared file	Accepts that a file both need cannot be split, so only one agent writes it ; the other sends a written request instead.	18% / 58%
6	coauthor-overlap	The exact merged code itself	Goes after the clash directly: they agree the shared code word-for-word, then both type the identical text .	78% / 69%

Why these six, and why in this order

- Slide 1 showed the clashes happen **inside** a file the agents had already talked about, on the **same line**. Talking about *files* is therefore too coarse to prevent them.
- That lets us **predict the result before running anything**: rules 1–4 only ever get the pair to agree at the level of files or intentions, so they should fail. Only rules 5–6 reach the actual lines of code, so only they should work.
- Rules 3 and 4 are the useful failures. **3** shows the agents were not being vague — they already named their files every single time. **4** shows the problem is not poor division of labour — forcing a real agreed split made the code more *correct*, but no easier to combine.

TAKEAWAY

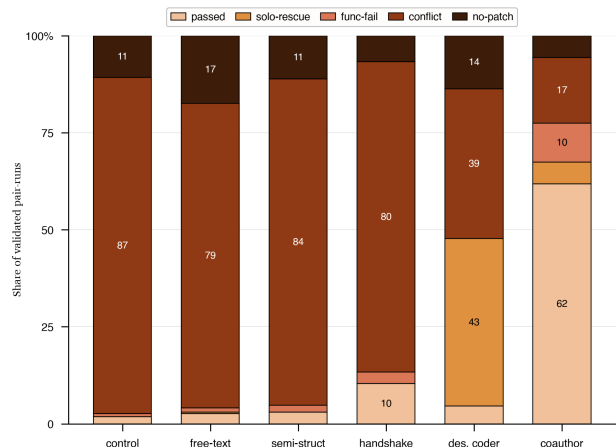
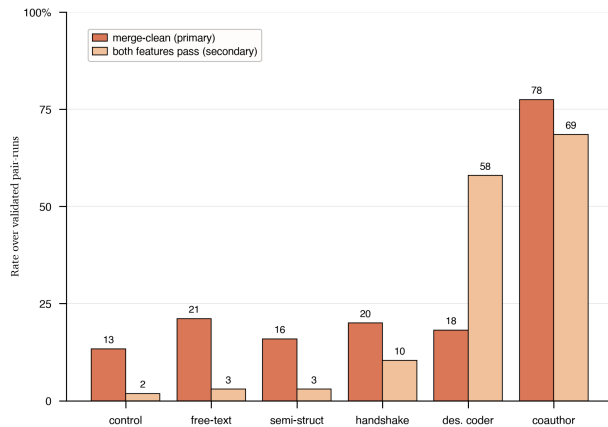
Each rung takes away one more thing the two agents can disagree about. The top rung takes away all of them.

What happened

18 tasks chosen because the two features genuinely overlap · ~1,250 attempts · Claude Sonnet 5

READING THE NUMBERS — the two things being measured

Combined cleanly	The two sets of edits went together automatically, with no clash. <i>The headline measure.</i>
Both features work	After combining, both agents' features actually pass their tests. <i>The stricter measure.</i>
Clash	Both agents rewrote the same lines, so the automatic combine refused.
One carried the pair	Both features work, but only because one agent's copy happened to contain both — not because the two combined.



How often each rule set worked. Taller is better. Five of the six sit in the same low band — only the last one (coauthor-overlap) breaks out. **How the attempts failed.** Each attempt in exactly one bucket. The dark band is clashes: it dominates everywhere except the last rule set.

Rule set	Combined cleanly	Both features work	Clashed
control	13%	2%	87%
free-text	21%	3%	79%
semi-structured	16%	3%	84%
plan-handshake	20%	10%	80%
designated-coder	18%	58%	39%
coauthor-overlap	78%	69%	17%

Key findings

- **The rules matter enormously — but only one set of rules worked.** Making the pair co-write the shared code took clean combining from **13% to 78%**, and both-features-working from **2% to 69%**. It is the only one that beats the baseline once you account for having tried six things.
- **Talking more achieved nothing.** Tidying up the messages, or planning who owns which file, left the result no better than giving them no way to talk at all — exactly as slide 1 predicted.
- **It works by changing what gets typed, not how much gets said.** The winning rule set produced **26** cases where both agents wrote byte-for-byte identical code; the other five produced **1** between them across more than a thousand attempts.
- **Working code and combinable code are different problems.** designated-coder gets both features working 58% of the time while still clashing as often as the baseline — one agent quietly carried the pair.
- **Still not solved.** Even the best rule set clashes 17% of the time, and another 10% combine cleanly but are broken. One AI model, one setup, one style of automatic combine.

TAKEAWAY

Rules for cooperation only help when they constrain **what the agents type** — not how much they say to each other.